Tile Engine ↔ Dispatcher Parity

Design & code walkthrough: what each file is and how it proves parity

The goal

The **dispatcher** selects and launches CK Tile GEMM kernels at runtime. **Tile Engine** is the existing offline codegen + benchmark system. Parity means: feed the same config to both and get the same kernel, the same registry key (computed offline during codegen *and* at runtime), the same numerical result, and — within a tolerance — the same performance.

The work is split so that everything provable **without a GPU** is proven on a CPU box (translation + the registry-key guarantee), while the GPU-only half (build, run, verify, benchmark) is staged and runs unchanged on a GPU node.

Pipeline at a glance

```
Tile Engine config JSON
|
v
te_to_dispatcher.py -----------------> dispatcher config objects (a)
| |
| identifier.py (Python encode_identifier)
| cpp_identifier_oracle (C++ KernelKey::encode_identifier)
| |
| check_identifier_parity.py (b) g++ only, NO GPU
|
|-- drive_codegen.py -> unified_gemm_codegen.py -> gemm_<id>.hpp (c)
| |
| harness.cpp + build_harness.sh (d) hipcc; run=GPU
|
'-- check_parity.py ------------------------------------------> (e)(f)
stage 1 identifier (always, CPU)
stage 2 numerical (GPU-gated)
stage 3 performance (GPU-gated)
```

The six deliverables

#	Deliverable	Files	Needs GPU?
a	Translator: TE JSON → dispatcher config objects	te_to_dispatcher.py	no
b	Kernel identifier matches codegen ↔ runtime	identifier.py, cpp_identifier_oracle.cpp, check_identifier_parity.py	no (g++)
c	Drive codegen for a single config	drive_codegen.py	no
d	Minimal C++ harness to run one kernel	harness.cpp, build_harness.sh	build no / run yes
e	Parity checker: dispatcher vs Tile Engine	check_parity.py	gated
f	Numerical parity first, then performance	check_parity.py (3-stage)	yes

File by file

te_to_dispatcher.py — the translator (a)

Reads a Tile Engine config JSON and emits one dispatcher config dict per valid *(tile × trait)* combination. Each dict has three parts:

- `_te` — the raw TE trait strings (`compv3 / intrawave / default`) kept verbatim, because codegen wants those names, not the canonical dispatcher forms.
- `signature` — the dispatcher `KernelKey` signature (dtypes, layouts, split_k, elementwise op…), already in canonical `to_string()` form.
- `algorithm` — tile/warp sizes, pipeline, scheduler, epilogue, pad flags, persistent, block_size.

It applies the TE→dispatcher mapping exactly **once** (e.g. scheduler `default`→`auto`, fp8/bf8 output→fp16, int8 acc→int32) and drops trait combos CK Tile does not support. Because the mapping happens here and only here, every downstream identifier is pure concatenation — which is what makes Python/C++ agreement provable.

identifier.py — Python identifier oracle (b)

Reproduces C++ `KernelKey::encode_identifier()` byte-for-byte from a config dict. It is deliberately dumb: no mapping, just concatenation in the exact field order of the C++ source, with optional suffixes (`_splitk{n}, _{op}, _d{n}, _sparse, _preshuffle`) emitted in the same order. `block_size` is intentionally omitted — it is part of equality but not the identifier.

```
dtype_a _ layoutABC _ pipeline _ epilogue _ scheduler _
padM _ padN _ padK _ persistent _
TMxTNxTK _ WMxWNxWK _ WTMxWTNxWTK [+ optional suffixes]
```

cpp_identifier_oracle.cpp — C++ identifier oracle (b)

Includes the real `kernel_key.hpp` and calls the *actual runtime* `encode_identifier()`. It reads flat `key=value` lines from stdin, rebuilds a `KernelKey` via the same `string_to_*` helpers the runtime uses, and prints the identifier. Configs are batched with a `---` separator so one process handles thousands of configs (essential at 283,968 configs). `kernel_key.hpp` is pure host C++, so this builds with g++ alone — no GPU, no hipcc, no cmake.

check_identifier_parity.py — the diff (b)

Translates the TE JSON, compiles the C++ oracle if stale, runs every config through both oracles, and asserts they match byte-for-byte. If they agree, the registry key computed offline during codegen equals the one computed at runtime, so dispatch lookups **cannot silently miss**. Result on the full default config: **283968 / 283968 match**.

drive_codegen.py — single-config codegen (c)

Picks one translated config (by `--index`) and invokes the dispatcher's `unified_gemm_codegen.py` to emit exactly that one kernel header. The subtlety: codegen's `--config` path expects each tile/trait parameter to be a *flat single-element list* (it iterates `tc["tile_m"]` directly), so we rebuild a minimal one-value-per-parameter config using the raw `_te` strings. The expected registry identifier is printed so the harness can find the generated `Kernel_<id>` struct.

harness.cpp + build_harness.sh — the single-kernel runner (d)

`harness.cpp` runs exactly one generated kernel through the `CK_TILE_SINGLE_KERNEL_INCLUDE` contract: defining that macro before including a generated header exposes a global `SelectedKernel` (with static `launch()`), the A/B/C data types, and `KERNEL_NAME`. The header path is injected at compile time as `PARITY_KERNEL_HEADER`, so one .cpp drives whichever kernel codegen produced.

- Builds an `rcr` fp16 GEMM (A row-major stride K, B col-major stride K, C row-major stride N) with deterministic inputs.
- Verifies against a CPU fp32 reference; tolerance scales as `1e-2 · sqrt(K)`; prints `PASSED / FAILED`.
- An unsupported argument throws → caught and reported as `SKIPPED` (a skip, not a failure).
- Reports kernel time and GFLOP/s when timing is on.

`build_harness.sh` compiles it with `hipcc --offload-arch=<gfx>` against the CK include tree. It builds on a CPU box; only *running* needs a GPU.

check_parity.py — the orchestrator (e, f)

Three escalating stages, in the required order:

- **Stage 1 — identifier** (always runs, CPU): delegates to check_identifier_parity. The offline↔runtime key guarantee.
- **Stage 2 — numerical** (GPU-gated): codegen → build harness → run `-verify=1` over several sizes and require `PASSED`. With `--te-build-dir`, the matching `benchmark_gemm_universal_<name>` is also run with verify; both must agree against the same reference.
- **Stage 3 — performance** (GPU-gated): `|disp - te| / te ≤ --perf-tol` (default 2%). Median of 10 outer runs per size; harness GFLOP/s converted to TFLOP/s to match Tile Engine's units.

A numerical failure short-circuits before performance is judged — enforcing “numerical first, then performance.” Without a GPU, stages 2–3 report `SKIPPED` (not FAILED) and the run still exits 0 if the identifier stage passed. `--dry-run` prints the full command plan without executing anything.

Two kinds of name — the key subtlety

There are **two** distinct names and the orchestrator uses each deliberately:

Name	Form	Built by	Used for
Registry identifier	canonical (scheduler `default`→`auto`)	encode_identifier()	dispatch lookup; Stage 1 proves it matches

Kernel / file name	raw TE strings (`default` stays `default`)	te_kernel_name()	names `gemm_<name>.hpp` and `benchmark_gemm_universal_<name>`

They coincide for the `fp16_rcr…intrawave` example but diverge whenever a TE string maps to a different canonical form. Conflating them would make the orchestrator look for a header/executable that does not exist.

Why this is trustworthy

- The C++ oracle calls the *real* runtime `encode_identifier()` — not a re-implementation — so a match is a genuine codegen↔runtime guarantee.
- The mapping lives in exactly one place (the translator), so identifiers are pure concatenation and the Python/C++ agreement is mechanical.
- Numerical parity is adjudicated against an independent CPU reference on both sides; performance parity is a relative tolerance with explicit units.
- GPU-gating is honest: missing GPU is SKIPPED, not silently PASSED; `rocminfo` is authoritative (a bare `/dev/kfd` is treated as no GPU).